

Doc. no.	P1209R0
Date:	2018-10-04
Project:	Programming Language C++
Reply to:	Alisdair Meredith < ameredith1@bloomberg.net >
	Stephan T. Lavavej < stl@microsoft.com >
Audience:	Library Evolution

Adopt Consistent Container Erasure from Library Fundamentals 2 for C++20

Revision History

Revision 0

Original version of the paper for the 2018 pre-San Diego mailing.

Table of Contents

- 0. [Revision History](#)
 - [Revision 0](#)
- 1. [Introduction](#)
- 2. [Problems to be addressed](#)
- 3. [Proposed Resolution](#)
 - [Annotated Original Wording](#)
- 4. [Alternatives Considered](#)
- 5. [Implementation Experience](#)
- 6. [Formal Wording](#)
- 7. [Acknowledgements](#)
- 8. [References](#)

1. Introduction

This paper proposes landing the consistent container erasure APIs from the Library Fundamentals v2 TS into C++20.

2. Problems to be addressed

Erasing members of a container that satisfy some condition seems like it would be an easy problem to solve, yet it contains many subtle traps for the careless and unwary. The simplest correct pattern involves walking the container, and *either* erasing an element to produce the next iterator *or* incrementing the current iterator to advance to the next element. This simple code yields errors far too often, while providing poor performance on popular sequence containers. The appropriate pattern for those sequence containers is quite different, applying the standard algorithm `remove_if` before calling the member `erase` to prune the tail of the sequence. However, this method is not the most appropriate choice (or even applicable) for other containers.

A more extensive rationale for why this problem was deemed worth solving for the Fundamentals TS can be found in the paper [N4009](#).

3. Proposed Resolution

The proposal is to land the wording from the Library Fundamentals TS directly. This is slightly tweaked, as the wording in the Fundamentals TS is mostly editorial guidance, so the words cannot be transcribed precisely as laid out in the TS. Instead, we apply that editorial guidance to the current working paper; see the [Formal Wording](#) below.

3.1 Annotated Original Wording

Below is the latest wording from the Fundamentals TS (v3) for this feature, marked up with diff-marks that correspond to the edits made while moving this wording directly into the affected standard clauses. The ~~strikeout~~ markup indicates wording that is dropped as part of the transition of landing this proposal into the main standard. Mostly it refers to removing the `experimental` headers and namespace, but also some commentary does not make the transition.

Not marked up is the renaming of template parameters to match the naming convention in the corresponding standard clause. One final tiny tweak, that is not marked up, is that `erase` will now precede `erase_if` in the formal wording, even in the case that `erase` is specified in terms of `erase_if`. This follows existing library precedent that the 'simpler' identifier goes first.

Also not marked up is the addition of a feature-test macro (present in the [Formal Wording](#) below).

6 Containers

[container]

6.1 Uniform container erasure

[container.erase]

6.1.1 Header synopsis

[container.erase.syn]

¹ ~~For brevity, this section specifies the contents of 9 headers, each of which behaves as described by 1.3.~~

```
namespace std::experimental {  
inline namespace fundamentals_v3 {  
  
// 6.1.2, Function template erase_if  
// 6.1.3, Function template erase  
  
// <experimental/string>  
template <class charT, class traits, class A, class Predicate>  
    void erase_if(basic_string<charT, traits, A>& c, Predicate pred);  
template <class charT, class traits, class A, class U>  
    void erase(basic_string<charT, traits, A>& c, const U& value);  
  
// <experimental/deque>  
template <class T, class A, class Predicate>  
    void erase_if(deque<T, A>& c, Predicate pred);  
template <class T, class A, class U>  
    void erase(deque<T, A>& c, const U& value);  
  
// <experimental/vector>  
template <class T, class A, class Predicate>  
    void erase_if(vector<T, A>& c, Predicate pred);  
template <class T, class A, class U>  
    void erase(vector<T, A>& c, const U& value);  
  
// <experimental/forward_list>  
template <class T, class A, class Predicate>  
    void erase_if(forward_list<T, A>& c, Predicate pred);  
template <class T, class A, class U>  
    void erase(forward_list<T, A>& c, const U& value);  
  
// <experimental/list>  
template <class T, class A, class Predicate>  
    void erase_if(list<T, A>& c, Predicate pred);  
template <class T, class A, class U>  
    void erase(list<T, A>& c, const U& value);  
  
// <experimental/map>  
template <class K, class T, class C, class A, class Predicate>  
    void erase_if(map<K, T, C, A>& c, Predicate pred);  
template <class K, class T, class C, class A, class Predicate>  
    void erase_if(multimap<K, T, C, A>& c, Predicate pred);  
  
// <experimental/set>  
template <class K, class C, class A, class Predicate>  
    void erase_if(set<K, C, A>& c, Predicate pred);  
template <class K, class C, class A, class Predicate>  
    void erase_if(multiset<K, C, A>& c, Predicate pred);
```

```

// <experimental/unordered_map>
template <class K, class T, class H, class P, class A, class Predicate>
    void erase_if(unordered_map<K, T, H, P, A>& c, Predicate pred);
template <class K, class T, class H, class P, class A, class Predicate>
    void erase_if(unordered_multimap<K, T, H, P, A>& c, Predicate pred);

// <experimental/unordered_set>
template <class K, class H, class P, class A, class Predicate>
    void erase_if(unordered_set<K, H, P, A>& c, Predicate pred);
template <class K, class H, class P, class A, class Predicate>
    void erase_if(unordered_multiset<K, H, P, A>& c, Predicate pred);

} // inline namespace fundamentals_v3
} // namespace std++experimental

```

6.1.2 Function template erase_if

[\[container.eraser.erase_if\]](#)

- 1 template <class charT, class traits, class A, class Predicate>
 void erase_if(basic_string<charT, traits, A>& c, Predicate pred);
 template <class T, class A, class Predicate>
 void erase_if(deque<T, A>& c, Predicate pred);
 template <class T, class A, class Predicate>
 void erase_if(vector<T, A>& c, Predicate pred);
 - 2 *Effects:* Equivalent to: `c.erase(remove_if(c.begin(), c.end(), pred), c.end());`
- 3 template <class T, class A, class Predicate>
 void erase_if(forward_list<T, A>& c, Predicate pred);
 template <class T, class A, class Predicate>
 void erase_if(list<T, A>& c, Predicate pred);
 - 4 *Effects:* Equivalent to: `c.remove_if(pred);`
- 5 template <class K, class T, class C, class A, class Predicate>
 void erase_if(map<K, T, C, A>& c, Predicate pred);
 template <class K, class T, class C, class A, class Predicate>
 void erase_if(multimap<K, T, C, A>& c, Predicate pred);
 template <class K, class C, class A, class Predicate>
 void erase_if(set<K, C, A>& c, Predicate pred);
 template <class K, class C, class A, class Predicate>
 void erase_if(multiset<K, C, A>& c, Predicate pred);
 template <class K, class T, class H, class P, class A, class Predicate>
 void erase_if(unordered_map<K, T, H, P, A>& c, Predicate pred);
 template <class K, class T, class H, class P, class A, class Predicate>
 void erase_if(unordered_multimap<K, T, H, P, A>& c, Predicate pred);
 template <class K, class H, class P, class A, class Predicate>
 void erase_if(unordered_set<K, H, P, A>& c, Predicate pred);
 template <class K, class H, class P, class A, class Predicate>
 void erase_if(unordered_multiset<K, H, P, A>& c, Predicate pred);
 - 6 *Effects:* Equivalent to:


```

for (auto i = c.begin(), last = c.end(); i != last; ) {
    if (pred(*i)) {
        i = c.erase(i);
    } else {
        ++i;
    }
}

```

6.1.3 Function template erase

[\[container.erasure.erase\]](#)

```
1 template <class charT, class traits, class A, class U>
   void erase(basic_string<charT, traits, A>& c, const U& value);
   template <class T, class A, class U>
   void erase(deque<T, A>& c, const U& value);
   template <class T, class A, class U>
   void erase(vector<T, A>& c, const U& value);

2 Effects: Equivalent to: c.erase(remove(c.begin(), c.end(), value), c.end());

3 template <class T, class A, class U>
   void erase(forward_list<T, A>& c, const U& value);
   template <class T, class A, class U>
   void erase(list<T, A>& c, const U& value);

4 Effects: Equivalent to: erase_if(c, [&](auto& elem) { return elem == value; });
```

~~[Note: Overloads of erase() for associative containers and unordered associative containers are intentionally not provided. —end note]~~

4. Alternatives Considered

A couple of design points were considered and rejected for this proposal.

The first idea is that these operations should be member functions, rather than free functions. This is rejected as we desire a level of uniformity of interface that would not be cleanly expressed in the container requirements tables, which are the closest notion of a container concept that we have. `std::array` would be a sequence container not supporting these operations, adding yet more special cases to a taxed special case. The requirements would need to be at the top of the hierarchy, under Container Requirements, or we end up repeating them for Sequence, Associative, and Unordered Associative Container Requirements, and the interface itself is not particularly simpler, while a free function extends and adapts user supplied containers more easily. Additionally, `erase` is already heavily overloaded as a member function; it would be confusing to further overload `c.erase(position)`, `c.erase(first, last)`, and `assoc.erase(key)`. Finally, these operations gain nothing from private access to the container, being expressible entirely through the public API with no loss of efficiency, so we prefer a coupling no stronger than is necessary.

The second idea is that these operations are acting on a range, and might more properly belong as part of the Ranges library proposal. That notion is rejected as these are more properly Container operations, that rely on container ownership semantics, and control the lifetime of elements and size on containers in ways that are inappropriate for the ranges library.

5. Implementation Experience

N4273 was implemented in VS 2015 RTM and GCC 6.1.

6. Formal Wording

This paper applies edits against [N4762](#).

16 Language support library

[\[language.support\]](#)

16.3.1 General

[\[support.limits.general\]](#)

Macro name	Value	Header(s)
...	...	...
__cpp_lib_enable_shared_from_this	201603L	<memory>
<u>__cpp_lib_erase_if</u>	<u>201811L</u>	<u><string></u> <u><deque></u> <u><forward_list></u> <u><list></u> <u><vector></u> <u><map></u> <u><set></u> <u><unordered_map></u> <u><unordered_set></u>
__cpp_lib_exchange_function	201304L	<utility>
...	...	...

20 Strings Library

[strings]

20.3.1 Header <string> synopsis

[string.syn]

```
#include <initializer_list>

namespace std {

    // 20.2, character traits
    template<class charT> struct char_traits;
    template<> struct char_traits<char>;
    template<> struct char_traits<char16_t>;
    template<> struct char_traits<char32_t>;
    template<> struct char_traits<wchar_t>;

    // 20.3.2, basic_string
    template<class charT, class traits = char_traits<charT>, class Allocator = allocator<charT>>
        class basic_string;

    ...

    // 20.3.3.9, inserters and extractors
    ...
    template<class charT, class traits, class Allocator>
    basic_istream<charT, traits>&
        getline(basic_istream<charT, traits>&& is,
                basic_string<charT, traits, Allocator>& str);

    // 20.3.3.X, string erasure
    template<class charT, class traits, class Allocator, class U>
    void erase(basic_string<charT, traits, Allocator>& c, const U& value);
    template<class charT, class traits, class Allocator, class Predicate>
    void erase_if(basic_string<charT, traits, Allocator>& c, Predicate pred);

    // basic_string typedef names
    using string = basic_string<char>;
    using u16string = basic_string<char16_t>;
    using u32string = basic_string<char32_t>;
    using wstring = basic_string<wchar_t>;

    // 20.3.4, numeric conversions
    ...

} // namespace std
```

20.3.3.X String Erasure

[string.erase]

```
template <class charT, class traits, class Allocator, class U>
void erase(basic_string<charT, traits, Allocator>& c, const U& value);
```

¹ *Effects:* Equivalent to: c.erase(remove(c.begin(), c.end(), value), c.end());;

```
template <class charT, class traits, class Allocator, class Predicate>  
void erase_if(basic_string<charT, traits, Allocator>& c, Predicate pred);
```

² *Effects:* Equivalent to: c.erase(remove_if(c.begin(), c.end(), _pred), c.end());;

21 Containers Library

[\[containers\]](#)

21.3.3 Header <deque> synopsis

[\[deque.syn\]](#)

```
#include <initializer_list>

namespace std {
    // 21.3.8, class template deque
    template<class T, class Allocator = allocator<T>> class deque;

    ...

    template<class T, class Allocator>
        void swap(deque<T, Allocator>& x, deque<T, Allocator>& y)
            noexcept(noexcept(x.swap(y)));

    template <class T, class Allocator, class U>
        void erase(deque<T, Allocator>& c, const U& value);
    template <class T, class Allocator, class Predicate>
        void erase_if(deque<T, Allocator>& c, Predicate pred);

    namespace pmr {
        template<class T>
            using deque = std::deque<T, polymorphic_allocator<T>>;
    }
} // namespace std
```

21.3.4 Header <forward_list> synopsis

[\[forward_list.syn\]](#)

```
#include <initializer_list>

namespace std {
    // 21.3.9, class template forward_list
    template<class T, class Allocator = allocator<T>> class forward_list;

    ...

    template<class T, class Allocator>
        void swap(forward_list<T, Allocator>& x, forward_list<T, Allocator>& y)
            noexcept(noexcept(x.swap(y)));

    template <class T, class Allocator, class U>
        void erase(forward_list<T, Allocator>& c, const U& value);
    template <class T, class Allocator, class Predicate>
        void erase_if(forward_list<T, Allocator>& c, Predicate pred);

    namespace pmr {
        template<class T>
            using forward_list = std::forward_list<T, polymorphic_allocator<T>>;
    }
} // namespace std
```

21.3.5 Header <list> synopsis

[\[list.syn\]](#)

```
#include <initializer_list>

namespace std {
    // 21.3.10, class template list
    template<class T, class Allocator = allocator<T>> class list;
    ...

    template<class T, class Allocator>
        void swap(list<T, Allocator>& x, list<T, Allocator>& y)
            noexcept(noexcept(x.swap(y)));

    template <class T, class Allocator, class U>
    void erase(list<T, Allocator>& c, const U& value);
    template <class T, class Allocator, class Predicate>
    void erase_if(list<T, Allocator>& c, Predicate pred);

    namespace pmr {
        template<class T>
            using list = std::list<T, polymorphic_allocator<T>>;
    }
} // namespace std
```

21.3.6 Header <vector> synopsis

[\[vector.syn\]](#)

```
#include <initializer_list>

namespace std {
    // 21.3.11, class template vector
    template<class T, class Allocator = allocator<T>> class vector;
    ...

    template<class T, class Allocator>
        void swap(vector<T, Allocator>& x, vector<T, Allocator>& y)
            noexcept(noexcept(x.swap(y)));

    template <class T, class Allocator, class U>
    void erase(vector<T, Allocator>& c, const U& value);
    template <class T, class Allocator, class Predicate>
    void erase_if(vector<T, Allocator>& c, Predicate pred);

    // 21.3.12, class vector<bool>
    template<class Allocator> class vector<bool, Allocator>;

    // hash support
    template<class T> struct hash;
    template<class Allocator> struct hash<vector<bool, Allocator>>;

    namespace pmr {
        template<class T>
            using vector = std::vector<T, polymorphic_allocator<T>>;
    }
} // namespace std
```

21.3.8.X Deque Erasure

[deque.erase]

```
template <class T, class Allocator, class U>
void erase(deque<T, Allocator>& c, const U& value);
```

¹ Effects: Equivalent to: `c.erase(remove(c.begin(), c.end(), value), c.end());`;

```
template <class T, class Allocator, class Predicate>
void erase_if(deque<T, Allocator>& c, Predicate pred);
```

² Effects: Equivalent to: `c.erase(remove_if(c.begin(), c.end(), pred), c.end());`;

21.3.9.X Forward List Erasure

[forward_list.erase]

```
template <class T, class Allocator, class U>
void erase(forward_list<T, Allocator>& c, const U& value);
```

¹ Effects: Equivalent to: `erase_if(c, [&](auto& elem) { return elem == value; })`;

```
template <class T, class Allocator, class Predicate>
void erase_if(forward_list<T, Allocator>& c, Predicate pred);
```

² Effects: Equivalent to: `c.remove_if(pred)`;

21.3.10.X List Erasure

[list.erase]

```
template <class T, class Allocator, class U>
void erase(list<T, Allocator>& c, const U& value);
```

¹ Effects: Equivalent to: `erase_if(c, [&](auto& elem) { return elem == value; })`;

```
template <class T, class Allocator, class Predicate>
void erase_if(list<T, Allocator>& c, Predicate pred);
```

² Effects: Equivalent to: `c.remove_if(pred)`;

21.3.11.X Vector Erasure

[vector.erase]

```
template <class T, class Allocator, class U>
void erase(vector<T, Allocator>& c, const U& value);
```

¹ Effects: Equivalent to: `c.erase(remove(c.begin(), c.end(), value), c.end());`;

```
template <class T, class Allocator, class Predicate>
void erase_if(vector<T, Allocator>& c, Predicate pred);
```

² Effects: Equivalent to: `c.erase(remove_if(c.begin(), c.end(), pred), c.end());`;

21.4.2 Header <map> synopsis

[associative.map.syn]

```
#include <initializer_list>
```

```
namespace std {
    // 21.4.4, class template map
    template<class Key, class T, class Compare = less<Key>,
            class Allocator = allocator<pair<const Key, T>>>
        class map;
```

```
...
```

```
template<class Key, class T, class Compare, class Allocator>
```

```

void swap(map<Key, T, Compare, Allocator>& x,
          map<Key, T, Compare, Allocator>& y)
    noexcept(noexcept(x.swap(y)));

template <class Key, class T, class Compare, class Allocator, class Predicate>
void erase_if(map<Key, T, Compare, Allocator>& c, Predicate pred);

// 21.4.5, class template multimap
template<class Key, class T, class Compare = less<Key>,
         class Allocator = allocator<pair<const Key, T>>>
    class multimap;

...

template<class Key, class T, class Compare, class Allocator>
    void swap(multimap<Key, T, Compare, Allocator>& x,
              multimap<Key, T, Compare, Allocator>& y)
        noexcept(noexcept(x.swap(y)));

template <class Key, class T, class Compare, class Allocator, class Predicate>
void erase_if(multimap<Key, T, Compare, Allocator>& c, Predicate pred);

namespace pmr {
    template<class Key, class T, class Compare = less<Key>>
        using map = std::map<Key, T, Compare,
                            polymorphic_allocator<pair<const Key, T>>>;
    template<class Key, class T, class Compare = less<Key>>
        using multimap = std::multimap<Key, T, Compare,
                                       polymorphic_allocator<pair<const Key, T>>>;
}
} // namespace std

```

21.4.3 Header <set> synopsis

[\[associative.set.syn\]](#)

```

#include <initializer_list>

namespace std {
    // 21.4.6, class template set
    template<class Key, class Compare = less<Key>, class Allocator = allocator<Key>>
        class set;

    ...

    template<class Key, class Compare, class Allocator>
        void swap(set<Key, Compare, Allocator>& x,
                  set<Key, Compare, Allocator>& y)
            noexcept(noexcept(x.swap(y)));

    template <class Key, class Compare, class Allocator, class Predicate>
    void erase_if(set<Key, Compare, Allocator>& c, Predicate pred);

    // 21.4.7, class template multiset
    template<class Key, class Compare = less<Key>, class Allocator = allocator<Key>>
        class multiset;

    ...

```

```
template<class Key, class Compare, class Allocator>
void swap(multiset<Key, Compare, Allocator>& x,
          multiset<Key, Compare, Allocator>& y)
noexcept(noexcept(x.swap(y)));
```

```
template <class Key, class Compare, class Allocator, class Predicate>
void erase_if(multiset<Key, Compare, Allocator>& c, Predicate pred);
```

```
namespace pmr {
    template<class Key, class Compare = less<Key>>
        using set = std::set<Key, Compare, polymorphic_allocator<Key>>;
    template<class Key, class Compare = less<Key>>
        using multiset = std::multiset<Key, Compare, polymorphic_allocator<Key>>;
}
} // namespace std
```

21.4.4.X Map Erasure

[\[map.erase\]](#)

```
template <class Key, class T, class Compare, class Allocator, class Predicate>
void erase_if(map<Key, T, Compare, Allocator>& c, Predicate pred);
```

¹ Effects: Equivalent to:

```
for (auto i = c.begin(), last = c.end(); i != last; _){
    if (pred(*i)) {
        i = c.erase(i);
    } else {
        ++i;
    }
}
```

21.4.5.X Multimap Erasure

[\[multimap.erase\]](#)

```
template <class Key, class T, class Compare, class Allocator, class Predicate>
void erase_if(multimap<Key, T, Compare, Allocator>& c, Predicate pred);
```

¹ Effects: Equivalent to:

```
for (auto i = c.begin(), last = c.end(); i != last; _){
    if (pred(*i)) {
        i = c.erase(i);
    } else {
        ++i;
    }
}
```

21.4.6.X Set Erasure

[\[set.erase\]](#)

```
template <class Key, class Compare, class Allocator, class Predicate>
void erase_if(set<Key, Compare, Allocator>& c, Predicate pred);
```

¹ Effects: Equivalent to:

```
for (auto i = c.begin(), last = c.end(); i != last; _){
    if (pred(*i)) {
        i = c.erase(i);
    } else {
        ++i;
    }
}
```

21.4.7.X Multiset Erasure

[multiset.erasure]

```
template <class Key, class Compare, class Allocator, class Predicate>  
void erase_if(multiset<Key, Compare, Allocator>& c, Predicate pred);
```

¹ *Effects:* Equivalent to:

```
for (auto i = c.begin(), last = c.end(); i != last; _){  
    if (pred(*i)){  
        i = c.erase(i);  
    } else {  
        ++i;  
    }  
}
```

21.5.2 Header <unordered_map> synopsis

[unord.map.syn]

```
#include <initializer_list>  
  
namespace std {  
    // 21.5.4, class template unordered_map  
    template<class Key,  
            class T,  
            class Hash = hash<Key>,  
            class Pred = equal_to<Key>,  
            class Alloc = allocator<pair<const Key, T>>>  
        class unordered_map;  
  
    // 21.5.5, class template unordered_multimap  
    template<class Key,  
            class T,  
            class Hash = hash<Key>,  
            class Pred = equal_to<Key>,  
            class Alloc = allocator<pair<const Key, T>>>  
        class unordered_multimap;  
  
    ...  
  
    template<class Key, class T, class Hash, class Pred, class Alloc>  
        void swap(unordered_map<Key, T, Hash, Pred, Alloc>& x,  
                unordered_map<Key, T, Hash, Pred, Alloc>& y)  
        noexcept(noexcept(x.swap(y)));  
  
    template<class Key, class T, class Hash, class Pred, class Alloc>  
        void swap(unordered_multimap<Key, T, Hash, Pred, Alloc>& x,  
                unordered_multimap<Key, T, Hash, Pred, Alloc>& y)  
        noexcept(noexcept(x.swap(y)));  
  
    template <class K, class T, class H, class P, class A, class Predicate>  
    void erase_if(unordered_map<K, T, H, P, A>& c, Predicate pred);  
  
    template <class K, class T, class H, class P, class A, class Predicate>  
    void erase_if(unordered_multimap<K, T, H, P, A>& c, Predicate pred);  
  
    namespace pmr {  
        template<class Key,  
                class T,  
                class Hash = hash<Key>,
```

```

        class Pred = equal_to<Key>>
using unordered_map =
    std::unordered_map<Key, T, Hash, Pred,
        polymorphic_allocator<pair<const Key, T>>>;

template<class Key,
        class T,
        class Hash = hash<Key>,
        class Pred = equal_to<Key>>
using unordered_multimap =
    std::unordered_multimap<Key, T, Hash, Pred,
        polymorphic_allocator<pair<const Key, T>>>;
}
} // namespace std

```

21.5.3 Header <unordered_set> synopsis

[\[unord.set.syn\]](#)

```

#include <initializer_list>

namespace std {
    // 21.5.6, class template unordered_set
    template<class Key,
            class Hash = hash<Key>,
            class Pred = equal_to<Key>,
            class Alloc = allocator<Key>>
    class unordered_set;

    // 21.5.7, class template unordered_multiset
    template<class Key,
            class Hash = hash<Key>,
            class Pred = equal_to<Key>,
            class Alloc = allocator<Key>>
    class unordered_multiset;

    ...

    template<class Key, class Hash, class Pred, class Alloc>
    void swap(unordered_set<Key, Hash, Pred, Alloc>& x,
            unordered_set<Key, Hash, Pred, Alloc>& y)
        noexcept(noexcept(x.swap(y)));

    template<class Key, class Hash, class Pred, class Alloc>
    void swap(unordered_multiset<Key, Hash, Pred, Alloc>& x,
            unordered_multiset<Key, Hash, Pred, Alloc>& y)
        noexcept(noexcept(x.swap(y)));

    template <class K, class H, class P, class A, class Predicate>
    void erase_if(unordered_set<K, H, P, A>& c, Predicate pred);

    template <class K, class H, class P, class A, class Predicate>
    void erase_if(unordered_multiset<K, H, P, A>& c, Predicate pred);

    namespace pmr {
        template<class Key,
                class Hash = hash<Key>,
                class Pred = equal_to<Key>>
        using unordered_set = std::unordered_set<Key, Hash, Pred,
            polymorphic_allocator<Key>>;
    }
}

```

```

template<class Key,
        class Hash = hash<Key>,
        class Pred = equal_to<Key>>
using unordered_multiset = std::unordered_multiset<Key, Hash, Pred,
                                                polymorphic_allocator<Key>>;
}
} // namespace std

```

21.5.4.X Unordered Map Erasure

[\[unord.map.erase\]](#)

```

template <class K, class T, class H, class P, class A, class Predicate>
void erase_if(unordered_map<K, T, H, P, A>& c, Predicate pred);

```

¹ *Effects:* Equivalent to:

```

for (auto i = c.begin(), last = c.end(); i != last; _){
    if (pred(*i)){
        i = c.erase(i);
    } else {
        ++i;
    }
}
}

```

21.5.5.X Unordered Multimap Erasure

[\[unord.multimap.erase\]](#)

```

template <class K, class T, class H, class P, class A, class Predicate>
void erase_if(unordered_multimap<K, T, H, P, A>& c, Predicate pred);

```

¹ *Effects:* Equivalent to:

```

for (auto i = c.begin(), last = c.end(); i != last; _){
    if (pred(*i)){
        i = c.erase(i);
    } else {
        ++i;
    }
}
}

```

21.5.6.X Unordered Set Erasure

[\[unord.set.erase\]](#)

```

template <class K, class H, class P, class A, class Predicate>
void erase_if(unordered_set<K, H, P, A>& c, Predicate pred);

```

¹ *Effects:* Equivalent to:

```

for (auto i = c.begin(), last = c.end(); i != last; _){
    if (pred(*i)){
        i = c.erase(i);
    } else {
        ++i;
    }
}
}

```

21.5.7.X Unordered Multiset Erasure

[[unord.multiset.erase](#)]

```
template <class K, class H, class P, class A, class Predicate>  
void erase_if(unordered_multiset<K, H, P, A>& c, Predicate pred);
```

¹ *Effects:* Equivalent to:

```
for (auto i = c.begin(), last = c.end(); i != last; _){  
    if (pred(*i)){  
        i = c.erase(i);  
    } else {  
        ++i;  
    }  
}
```

7. Acknowledgements

Thanks to Timur Doumler for encouraging this proposal.

8. References

- [N4009](#) Uniform Container Erasure, Stephan T. Lavavej
- [N4273](#) Uniform Container Erasure (Revision 2), Stephan T. Lavavej